

FOL2NS: Generating Natural Sentences from First-Order Logic

Abstract

Translating formal language into natural language is a foundational challenge in NLP, driving various downstream applications in semantic parsing, theorem validation and question-answering. In this study, I introduce First-order logic to Natural Sentence (FOL2NS), a neurosymbolic framework to provide synthetic FOL formulas as well as convert them into human expression. It covers a deeply nested structure with gradual quantifier depth (QD) that is rarely captured by existing corpora. By combining rule-driven modules with fine-tuning language models, FOL2NS extends the diversity and coverage of generated samples. In the experiments, I systematically evaluate the model’s ability with both character-level analysis and overall performance. Related results show that FOL2NS can reliably produce well-formed templates and fluent statements, but faces a challenge for more precise semantic representations and natural generation with increasing structural complexity.

1 Introduction

In recent years, Large Language Models (LLMs) have shown impressive performance in Natural Language Generation (NLG). Despite this progress, their ability to tackle logic-to-text tasks requires more exploration, especially when generating natural sentences from first-order logic (FOL). The demand for formal correctness is equally important as fluent, coherent texts in such tasks. To this end, I propose a novel FOL2NS translation pipeline based on formal syntax, which bridges the rigour of the symbolic system and the data-driven power of large pre-trained models. A related example of outputs in every step is shown in Table 1.

The pipeline construction comprises multiple steps: (i) generating mathematical formulas by context-free grammars, (ii) incorporating real-world lexical items into the formulas to get their logical forms, (iii) finetuning a pretrained T5 model

on a preprocessed dataset from existing corpus, and (iv) mapping the logic forms to natural sentences via this model. In theory, this structure can generate unlimited formulas and increase the overall sample size under explicit changes of length and depth. It enables on-demand adjustment of the maximum nesting level and seamless integration of new templates into the pipeline. Simultaneously, it facilitates precise control over the complexity of both generated FOL formulas and their corresponding natural sentences. Related source code is available at: https://github.com/Mifuxuanan/LELA70502_FOL2NS-Generating-Natural-Language-Sentences-from-First-Order-Logic

2 Related Work

Previous work has connected natural language with logical representations and provides parallel or semantically aligned translation forms. On one hand, for NLG from structured tables, [Chen et al. \(2020\)](#) introduce Logic2Text, a large-scale dataset of 10,753 paired logical forms and descriptions. They employ several models from rule-based templates, encoder-decoder architectures to fine-tuned GPT-2, for controllable and high-fidelity NLG. Differently, my work focuses on generating texts from first-order logic (FOL) through fine-tuning the T5 model with different strategies.

On the other hand, there are several studies about Natural Language(NL) to FOL translation. LogicNLI ([Tian et al., 2021](#)) offers a large-scale natural language inference (NLI) dataset for multi-step FOL reasoning with 20k semi-synthetic examples. FOLIO ([Han et al., 2024](#)) contains 1,430 human-annotated natural language reasoning examples with verified FOL translations. NL2FOL ([Lalwani et al., 2025](#)) presents a neurosymbolic pipeline that autoformalizes natural language arguments into FOL through LLM-driven decomposition and SMT validation. However, these stud-

Type	Output
FOL Formula	$\forall a(A(a) \rightarrow \forall b(\neg B(b, c) \vee \neg C(b)))$
Lexicalized FOL	$\forall a(HasOfficeIn(a) \rightarrow \forall chef(\neg LivesIn(chef, zone) \vee \neg IsThoughtful(chef)))$
Natural Sentence	If the chef has an office in a building, then the chef neither lives in the zone nor is thoughtful.

Table 1: An output example in every step of constructing FOL2NS. It starts with a FOL formula with the quantifier depth of two in both mathematical and lexicalised formats. The corresponding natural sentence is generated by fine-tuning the T5-large model.

ies only exhibit the logical form as the translation outputs and don’t consider the nesting levels controlled by quantifiers in FOL formulas. In contrast, my work starts with the initial mathematical formulas and highlights every step of how to map rules with quantifier depths to natural sentences. It also covers the T5 model’s text generation quality from multiple nested logical structures.

3 Definition

3.1 First-order logic

First-order logic (FOL) is a logical system for reasoning about the properties of objects or the relationships between objects in a domain. This kind of proposition consists of k-ary predicate symbols, variables, constants and function symbols. In FOL, the atomic formulas (e.g. $P(a)$, $Q(a, b)$) are predicates that assert a relationship among certain elements. Beyond that, compound formulas are assembled using more logical connectives, such as negation ($\neg$), implication ($\rightarrow$), exclusive-or ($\oplus$), conjunction ($\wedge$) and disjunction($\vee$).

3.2 Quantifier Depth

The most significant concept in FOL is quantification: the ability to assert that a certain property holds for all elements with universal quantifiers ($\forall$) or that it holds for some element with existential quantifiers ($\exists$). "Quantifier" is the key feature that sets FOL and other propositional logics apart. To measure the structural complexity of a FOL formula, one often considers its quantifier depth (QD). Given the formula F and formula G, the related relationships are defined as follows:

1. An atomic formula has $QD = 0$;
2. Negation does not increase the depth: $QD(\neg F) = QD(F)$;
3. For binary connectives, the depth is the maximum depth between the sides of operands: $QD(F \wedge G) = QD(F \vee G) = \max(QD(F), QD(G))$;

4. Each application of a quantifier increases the depth by one: $QD(\forall x F) = QD(\exists x F) = QD(F) + 1$.

4 Data Construction

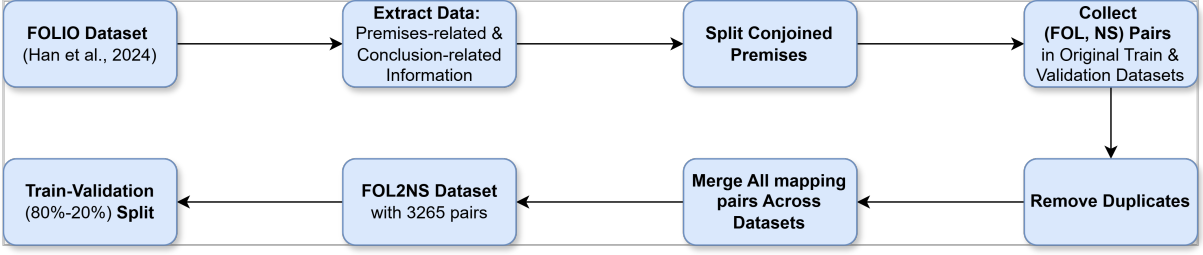
The dataset for fine-tuning models is constructed in 2 stages as illustrated in Figure 1. The source of Training and Validation sets is FOLIO (Han et al., 2024), an expert-written dataset designed for FOL reasoning. Despite its rich logical and semantic complexity, it has a distinct limitation in the sample size. Based on the original 1,430 samples of FOLIO, I expanded thousands of test samples through synthetic FOL formulas. This approach combines the strengths of both to systematically construct a diverse and controllable dataset.

4.1 FOLIO Dataset

Han et al. (2024) present a novel NL-to-FOL translation benchmark. It is inherently equipped with parallel pairs between natural-language statements and their FOL representations. Every premise and conclusion pair has been transcribed into FOL format and automatically verified by an FOL inference engine, ensuring their logical correctness. Compared to other synthetic datasets, each example in FOLIO is mostly aligned with a real-world background with increased reasoning complexity and a larger vocabulary of 4351 size.

The FOLIO dataset contains 1,001 training and 203 validation instances drawn from 487 original “stories”, where each data point is packaged as seven columns, including paired logical and NL representations of premises and conclusions, three classification labels and information about their id (Appendix Figure 3). It also involves various logic patterns with special symbols ($\forall$, $\neg$, $\wedge$, $\vee$). However, the cases of existential quantification ($\exists$) and initial FOL formulas are not present in this dataset, which are complemented in my Test dataset construction.

Module A: Preprocessing FOLIO



Module B: FOL2NW

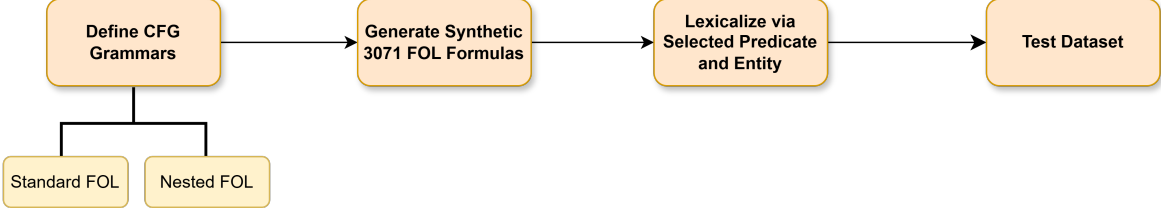


Figure 1: Overview of the proposed framework for dataset construction. Module A gets the Train and Validation datasets by cleaning the FOLIO dataset. Module B generates the Test dataset by CFG rules and FOL-to-Natural-Words mapping.

4.2 Module A: Preprocessing FOLIO

The data cleaning of FOLIO is conducted through several steps. I began with extracting the premise-related and conclusion-related components from each sample. The conjoined premises were split into individual clauses so that each natural sentence (NS) aligned clearly with a single logical form. Then, by reversing the mapping order in FOLIO, I collected all resulting pairs of FOL and NS from the original datasets. After removing duplicate pairs, this process yielded 3,625 formula-statement pairs and was finally partitioned into an 80%-20% Training-Validation split for subsequent fine-tuning. The related statistical information is shown in Figure 2 and Table 2. The token frequency distributions for inputs (FOL) share an analogous shape as those for outputs (NS). In all cases, the smaller Kullback-Leibler divergence $D_{KL}(validation||train)$ ensures that the model can see both high-frequency tokens and tail-end vocabulary across splits for higher-quality generation.

4.3 Module B: FOL2NW

Constructing the test data involves creating a synthetic set of FOL expressions and converting them into the lexicalized logical form. To increase structural complexity, the generation strategy contains two context-free grammars (CFG) over the logical connectives and atomic predicates. The de-

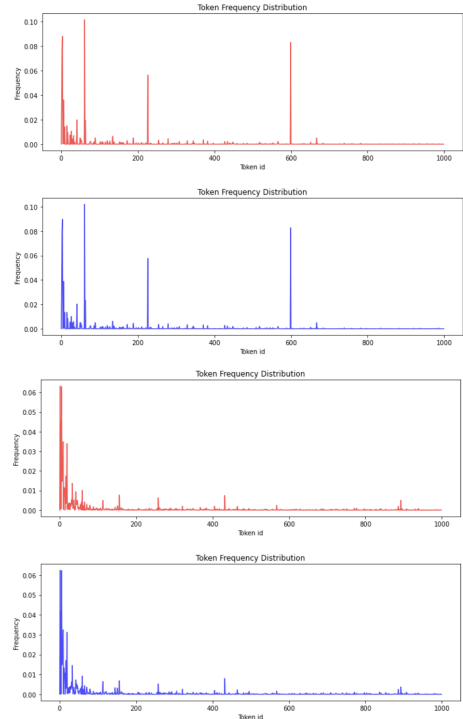


Figure 2: The token frequency distribution of FOL as inputs (above two) and NS as outputs (bottom two), for the preprocessed dataset of Module A in Training (red) and Validation (blue) splits.

Train (P) vs. Validation (Q)	KL Divergence	
	PQ	QP
Input	0.5094	0.2186
Output	1.1563	0.4896

Table 2: KL divergence of Train versus Validation sets in Module A.

Symbol	Lexical Item
Original Formula	$\forall b(\neg B(b, c))$
Sampling Predicate and Related Entity	<i>LivesIn</i> ([" <i>Person</i> ", " <i>Location</i> "])
Sampling Term in the Target Entity	" <i>Person</i> ": <i>chef</i> , " <i>Location</i> ": <i>zone</i>
Final Output	$\forall chef(\neg LivesIn(chef, zone))$

Table 3: A simplified example of FOL2NW workflow.

sign idea is based on the definition of FOL (§ 3.1) and QD (§ 3.2). For “Standard CFG”, it started from atomic formulas and repeatedly combined logical operators from left to right, resulting in flat, two-way branches. By adjusting the maximum depth, it was generated with increasing length but without nested predicate arguments. In addition to the above, “Nested CFG” permitted predicate symbols to take other formulas as arguments. Again, a global depth parameter limits term-argument nesting, which allows the control of quantifier depth and formula complexity at the same time. To align with the logic formatting used in FOLIO, I constrained the whole depth in the range of 4-10. I obtained 3,071 unique FOL formulas by uniformly sampling from both grammars. The related examples are shown in Appendix Figure 4.

The next step is to fill each abstract formula with lexical items and convert them into concrete, readable formats. First, I defined two vocabularies: a set of 50 predicate names, including both unary predicates (e.g., “*IsHappy*”) and binary predicates (e.g., “*Like*”), and a set of entities under seven classes (“*Person*”, “*Organization*”, “*Location*”, “*Field*”, “*Object*”, “*Animal*”, and “*Drink*”). Second, the lexicalized procedure was performed for every CFG-based formula. I uniformly sampled from the 50 predicates, and according to its arity, selected the random individual term in the related entity class. Third, each variable in the template was substituted by the corresponding sampled predicate or entity, except first variables did not change to keep the subject-predicate agreement in the statement. A simplified step-by-step

example is displayed in Table 3. By repeating the whole operation for all 3,071 samples, it produces a diverse test set of fully lexicalized FOL logical format.

5 Experimental Setups

5.1 Model

I selected vanilla pretrained T5 (Raffel et al., 2023) to fine-tune on the prepared dataset in Module A and tested it on the generated logical formats in Module B. Due to its text-to-text paradigm, the model allows us to take FOL as the source language and English as the target language without any additional architectural change. During inference, the flexible decoding strategies, such as beam search and repetition penalty, provide my work with various methods for high-quality generation.

Although T5 has been pre-trained on massive amounts of data, its original vocabulary does not contain tokens of logical operators and quantifiers. I tried to directly add all these special symbols in the tokenizing stage, but the model failed to recognize them and still needs enough training on this task. Therefore, I decided to replace these symbols with the corresponding lexical items referring to Table 4.

5.2 Fine-tuning Details

I consider different optimisation-related hyperparameters, including learning rates at 1e-4 and 1e-5, and training epochs of 20, 25 and 30. To ensure high-quality generation without excessive repetition or unnecessary parts, I also applied four decoding strategies: “Standard”, “Ad-

Symbol	Lexical Item
$\neg$	No
$\forall$	For All
$\exists$	There Exists
$\oplus$	XOR
$\rightarrow$	implies
$\wedge$	and
$\vee$	or

Table 4: The mapping relationship between symbols and lexical items.

justed”, “Prefixed” and “Adjusted with Length”. Each strategy represented a further optimization built upon the previous one. For “Standard”, I removed every padding token’s label by -100 to avoid appearing “ $< pad >$ ” in outputs. For “Adjusted”, I changed the arguments in “*model.generate*” with “*early_stopping*”, “*num_beams*” as 5, “*repetition_penalty*” as 1, and “*no_repeat_ngram_size*” as 2. For “Prefixed”, I added a prefix “Translate FOL formula to English:” to clarify the objective of the model. For “Adjusted with Length”, I set “max length” from the default value 20 to 64, which prevented the model from being prematurely truncated. Given training efficiency and cost, I tried T5-base with a batch size of 32 in initial experiments and compared different configurations. However, I finally decided to go ahead with T5-large with a batch size of 8 to gain better performance.

5.3 Performance Metrics

I used Levenshtein Distance and BLEU score for overall evaluation in the Validation stage. Shorter distance indicates closer alignment to the reference, while BLEU scores range from 0 to 1, with values closer to 1 representing higher generation quality. The related notations are defined as follow: for each valid example i , the model’s generated sentence is viewed as candidate $\hat{y}_i$ and its reference label is y_i . The number of all non-empty pairs is denoted as N .

Levenshtein Distance. Levenshtein Vladimir (1966) defined it as a string metric for measuring the difference between two sequences. Building on the standard definition, I apply a character-level edit distance metric, and denote $D(\hat{y}_i, y_i)$ as the minimum edit number to convert $\hat{y}_i$ into y_i . Beyond raw edit counts, the distance score is normalized by the maximum token length of the

pair $(\hat{y}_i, y_i)$. The Levenshtein distance d_i and related score s_i are calculated as follows:

$$d_i = D(\hat{y}_i, y_i) \quad (1)$$

$$s_i = \frac{D(\hat{y}_i, y_i)}{\max(|\hat{y}_i|, |y_i|)} \quad (2)$$

The average distance $\bar{D}$ and average normalized score $\bar{S}$ are reported as follows:

$$\bar{D} = \frac{1}{N} \sum_{i=1}^N d_i = \frac{1}{N} \sum_{i=1}^N D(\hat{y}_i, y_i) \quad (3)$$

$$\bar{S} = \frac{1}{N} \sum_{i=1}^N s_i = \frac{1}{N} \sum_{i=1}^N \frac{D(\hat{y}_i, y_i)}{\max(|\hat{y}_i|, |y_i|)} \quad (4)$$

BLEU Score. I adopt the original BLEU formulation of Papineni et al. (2002), but introduce a few adjustments to make it more robust in practice. The outer summation over multiple references reduces to a single sentence, as my case has only one gold label y_i per candidate $\hat{y}_i$. To keep numerical stability, I also add a epsilon ϵ of 10^{-3} to the denominator in clipped-count normalization and brevity penalty. The related formulas are calculated as follows:

$$p_n = \frac{\sum_{n\text{-gram} \in \text{cand}} \text{Count}_{\text{clip}}(n\text{-gram})}{\sum_{n\text{-gram}' \in \text{cand}} \text{Count}(n\text{-gram}') + \epsilon} \quad (5)$$

$$\text{BLEU}_i = \text{BP}_i \exp\left(\frac{1}{N} \sum_{n=1}^N \ln p_{n,i}\right) \quad (6)$$

$$\text{BP} = \begin{cases} 1, & c > r, \\ \exp\left(1 - \frac{r}{c+\epsilon}\right), & c \leq r, \end{cases} \quad \epsilon = 10^{-3} \quad (7)$$

The average BLEU score $\overline{\text{BLEU}}$ is reported as follows:

$$\overline{\text{BLEU}} = \frac{1}{N} \sum_{i=1}^N \text{BLEU}_i \quad (8)$$

Metric	Training Strategy			
	Standard	Adjusted	Prefixed	Adjusted with Length
Avg Edit Distance	31.99	27.83	26.98	22.97
Avg Distance Score	2.42	2.13	1.94	1.71
AVG BLEU	0.44	0.47	0.52	0.59

Table 5: Performance metrics with a learning rate of $1e - 4$ using different training strategies.

Metric	Learning Rate	
	$1e-4$	$1e-5$
Avg Edit Distance	27.83	44.78
Avg Distance Score	2.13	3.93
AVG BLEU	0.47	0.092

Table 6: Performance metrics across different learning rates using Standard training strategy.

Metric	Training Epoch		
	20	25	30
Avg Edit Distance	33.46	28.69	27.83
Avg Distance Score	2.64	2.21	2.13
AVG BLEU	0.36	0.45	0.47

Table 7: Performance metrics across different training epochs with a learning rate of $1e - 4$ using Adjusted training strategy.

6 Result

6.1 Hyperparameter Sensitivity Analysis

The experimental results reveal several important trends in how the model responds to different training strategies and hyperparameters. First, Table 5 demonstrates that each adjustment to decoding parameters or prompt prefixing contributes progressively to narrow the gap between generated sentences and reference labels. The single largest drop in edit distance of -4.16 comes from the Standard-to-Adjusted step. This phenomenon is consistent with the modified decoding settings in "*model.generate*", especially the increased beam width that allows the model to explore more possible high-quality continuations and improve the chance of selecting closer answers with reference sentences. Other parameters also prevent the outputs from redundant or looping phrases and repetitive bigram tokens. Additionally, the Adjusted-with-Length one yields the best performance across all metrics. The greatest improvement of BLEU (+0.07) also occurs in this step when changing from the Prefixed solution. Building on the strengths

of all previous strategies, it further addresses the issue of sequence-length mismatches. Taken together, these parameter adjustments lead to more concise, fluent generation and significantly enhance the model’s ability to parse logical forms and convert them into natural language.

Moreover, Table 6 shows that the model with a learning rate of $1e - 4$ significantly outperforms the case of $1e - 5$ for all metrics. In Table 7, I observe a steady gain when extending from 20 epochs to 25, and the performance peaks at 30 epochs with only marginal progress. Overall, these findings suggest that an “Adjusted with Length” strategy with $1e - 4$ as the learning rate provides the best output, while the number of epochs only has a slight effect on the final result. Under this configuration on T5-large with 20 epochs, it achieved optimal performance across all average metrics with an edit distance of 20.17, a normalized score of 1.46 and a BLEU of 0.67.

6.2 Error Analysis

I manually examine the first five candidate-reference pairs in the Validation set. According to Appendix Figure 6, the model performs perfectly on simple FOL formulas with only one predicate in examples 1, 4 and 5. However, when the logical templates become more complex, especially with multiple quantifiers or nested clauses, it struggle with two main errors. First, the model omits the intended contrastive relationship. In example 2, it does not map the exclusive logic to the corresponding connective “*but*”. Second, it tends to simplify noun phrases by dropping multi-word modifiers. The “*harmful things*” are reduced to “*harm*” in example 3. Moreover, example 3 shows the difference in letter cases, where the model replaces “*Acts*” with “*acts*”. Whether this should be regarded as an error is controversial, as “*acts*” is the correct lowercase form unless “*Acts*” represents a special term. Together, the model correctly captures the main logical relationships from the original formulas but fails to translate more precise

semantic meaning.

Additionally, Appendix Figure 7 shows that nearly all sentences generated by the model are grammatically correct English and read smoothly. It indicates that the model has the generalization ability to handle simple quantifiers and logical connectives and correctly map them in the out-of-domain dataset. However, when it comes to more complex patterns with multiple indicators, many outputs collapse the nested forms into simplified statements, missing the original logical scope. Less than 50% of examples fully preserve logical structures. Also, some outputs appear to have strange semantic meanings. The cases involving the XOR operator ($\oplus$) are frequently misunderstood as the meaning of "or". Consequently, the initial exclusive operator is reduced to parallel clauses, losing the critical distinction between exclusive and inclusive disjunctions. For this inconsistent processing, it may need a more precise definition to make the model understand when substituting the original symbol for English phrases.

7 Discussion

The T5 model on logic-to-text tasks achieves strong performance on in-domain distribution and can convert the synthetic logical forms into fluent natural sentences. Despite these strengths, it still remain several limitations.

First, my observation reveals that as quantifier depth gradually increases, sentences generated by the model become unnatural and affect the final translation quality. As a famous idea coming from Chomsky, human language allows for infinite recursion, but people can not produce or understand unlimited recursion due to memory constraints. Similarly, language models should be able to process nesting data up to their maximum layer depth, but struggle with converting extreme nested logical forms in practice.

Second, the semantic coherence of outputs heavily depends on the coverage and alignment of the predicate and entity vocabularies in Module B's test-set construction. Further research can consider prompt engineering in LLMs to select optimal predicate-entity pairs, thereby producing natural sentences with semantic richness and more real-world knowledge. This limitation also raises the ethical concerns of FOL2NS to some extent. If certain predicates or entities only appear in specific contexts, the model may learn to generate

biased descriptions. Future work should involve a double-check of all representations within the framework. Third, this study lacks true labels for the synthetic test set. It would be a wise choice to quantify the model's outputs by human evaluation or additional reference annotations. It can also replace validation loss with the BLEU score as an early stopping checkpoint to provide better task-oriented performance in the future.

8 Conclusion

For logical-to-text translation, FOL2NS ensures both formally correct and semantically diverse outputs in general. The model in the final stage achieves low edit distance and high BLEU scores on in-domain datasets, and maintains fluency when generating texts from synthetic logical forms, though it shows limitations in fine-grained semantics and extreme nesting. Future work will focus on prompt engineering to dynamically select predicate-variable pairs, better training and audit strategies, and human-annotated evaluation.

References

- Zhiyu Chen, Wenhui Chen, Hanwen Zha, Xiyu Zhou, Yunkai Zhang, Sairam Sundaresan, and William Yang Wang. 2020. [Logic2Text: High-Fidelity Natural Language Generation from Logical Forms](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2096–2111, Online. Association for Computational Linguistics.
- Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin. 2021. [Diagnosing the First-Order Logical Reasoning Ability Through LogicNLI](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3738–3747, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenxing Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alexander Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Maciej Kryscinski, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, Arman Cohan, and Dragomir Radev. 2024. [FOLIO: Natural Language Reasoning with First-Order Logic](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22017–22031, Miami, Florida, USA. Association for Computational Linguistics.

Abhinav Lalwani, Tasha Kim, Lovish Chopra, Christopher Hahn, Zhijing Jin, & Mrinmaya Sachan. 2025. [Autoformalizing Natural Language to First-Order Logic: A Case Study in Logical Fallacy Detection](#). arXiv preprint arXiv:2405.02318.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, & Peter J. Liu. 2023. [Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer](#). arXiv preprint arXiv:1910.10683.

Levenshtein, Vladimir I. (1966). [Binary codes capable of correcting deletions, insertions, and reversals](#). *Soviet Physics. Doklady*, 10(8), 707–710.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. [Bleu: a Method for Automatic Evaluation of Machine Translation](#). In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.

story_id	premises	premises-FOL	conclusion	conclusion-FOL	label	example_id	
0	406	All people who regularly drink coffee are dependent on caffeine.\nPeople regularly drink coffee, or they don't want to be addicted to caffeine, or both.\nNo one who doesn't want to be addicted to caffeine is unaware that caffeine is a drug.\nRina is either a student who is unaware that caffeine is a drug, or she is not a student and is she aware that caffeine is a drug.\nRina is either a student who is dependent on caffeine, or she is not a student and not dependent on caffeine.	$\forall x (\text{DrinkRegularly}(x, \text{coffee}) \rightarrow \text{IsDependentOn}(x, \text{caffeine})) \wedge \forall x (\text{DrinkRegularly}(x, \text{coffee}) \vee (\neg \text{WantToBeAddictedTo}(x, \text{caffeine})) \wedge \neg \text{AwareThatDrug}(x, \text{caffeine})) \wedge \neg (\text{Student}(\text{rina}) \oplus \neg \text{AwareThatDrug}(\text{rina}, \text{caffeine})) \wedge \neg (\text{IsDependentOn}(\text{rina}, \text{caffeine}) \oplus \text{Student}(\text{rina}))$	Rina doesn't want to be addicted to caffeine or is unaware that caffeine is a drug.	$\neg \text{WantToBeAddictedTo}(\text{rina}, \text{caffeine}) \vee (\neg \text{AwareThatDrug}(\text{rina}, \text{caffeine}))$	True	1126

Figure 3: An example of FOLIO.

Inputs	
0	$\forall a (A(a) \wedge \forall b B(b) \oplus \exists c C(c, d))$
1	$\forall a (\neg A(a) \oplus \neg B(a, b) \vee \neg C(a, c))$
2	$\forall a (A(a) \rightarrow \forall b (\neg B(b, c) \vee \neg C(b)))$
3	$\forall a A(a) \rightarrow \forall b \neg B(b) \wedge \forall c \neg C(c)$
4	$\forall a (A(a) \wedge \exists b (\neg B(b) \vee \exists c C(c, d)))$
5	$\forall a A(a) \rightarrow \exists b B(b) \wedge \forall c C(c)$
6	$\forall a (\neg A(a, b) \rightarrow B(a))$
7	$\forall a \neg A(a) \rightarrow \exists b B(b) \vee \exists c \neg C(c, d)$
8	$\forall a A(a, d) \wedge \exists b \neg B(b) \vee \exists c \neg C(c, e)$
9	$\forall a (\neg A(a) \rightarrow B(a) \oplus \neg C(a, b))$

Figure 4: The first 10 examples of defined FOL formulas in Module B.

FOL	FOL2NW
0 $\forall a (A(a) \wedge \forall b B(b) \oplus \exists c C(c, d))$	$\forall a (\text{IsCold}(a) \wedge \forall \text{scientist Visits}(\text{scientist}) \oplus \exists \text{citizen SpecializesIn}(\text{citizen}, \text{philosophy}))$
1 $\forall a (\neg A(a) \oplus \neg B(a, b) \vee \neg C(a, c))$	$\forall a (\neg \text{Visits}(a) \oplus \neg \text{Admires}(a, \text{customer}) \vee \neg \text{Sponsors}(a, \text{club}))$
2 $\forall a (A(a) \rightarrow \forall b (\neg B(b, c) \vee \neg C(b)))$	$\forall a (\text{Regulates}(a) \rightarrow \forall \text{institute } (\neg \text{IsSpacious}(\text{institute}, c) \vee \neg \text{Regulates}(\text{institute})))$
3 $\forall a A(a) \rightarrow \forall b \neg B(b) \wedge \forall c \neg C(c)$	$\forall a \text{Inspires}(a) \rightarrow \forall \text{citizen } \neg \text{IsGenerous}(\text{citizen}) \wedge \forall \text{bureau } \neg \text{Sells}(\text{bureau})$
4 $\forall a (A(a) \wedge \exists b (\neg B(b) \vee \exists c C(c, d)))$	$\forall a (\text{Fights}(a) \wedge \exists \text{trading company } (\neg \text{InvestsIn}(\text{trading company}) \vee \exists \text{specialist Adopts}(\text{specialist}, \text{bear})))$
5 $\forall a A(a) \rightarrow \exists b B(b) \wedge \forall c C(c)$	$\forall a \text{Develops}(a) \rightarrow \exists \text{province IsSpacious}(\text{province}) \wedge \forall \text{singer IsBusy}(\text{singer})$
6 $\forall a (\neg A(a, b) \rightarrow B(a))$	$\forall a (\neg \text{Pets}(a, \text{donkey}) \rightarrow \text{Produces}(a))$
7 $\forall a \neg A(a) \rightarrow \exists b B(b) \vee \exists c \neg C(c, d)$	$\forall a \neg \text{IsExcited}(a) \rightarrow \exists \text{executive Influences}(\text{executive}) \vee \exists \text{park } \neg \text{IsCrowded}(\text{park}, d)$
8 $\forall a A(a, d) \wedge \exists b \neg B(b) \vee \exists c \neg C(c, e)$	$\forall a \text{IsSick}(a, d) \wedge \exists \text{trader } \neg \text{Influences}(\text{trader}) \vee \exists \text{division } \neg \text{Awards}(\text{division}, \text{journalist})$
9 $\forall a (\neg A(a) \rightarrow B(a) \oplus \neg C(a, b))$	$\forall a (\neg \text{IsHot}(a) \rightarrow \text{Trusts}(a) \oplus \neg \text{OperatesIn}(a, \text{park}))$

Figure 5: Top 10 examples of the preprocessed FOL format with selected predicate-variable pairs.

The first five samples:

Input: No Musician(dan)
Output: Dan is not a musician.
Target: Dan is not a musician.
Distance: 0
BLEU: 0.9999545475205672

Input: For All x For All y (Student(x) and GetGradeIn(x, a, englishClass) implies EnglishClassScore(x, y) and GreaterThan90(y) and LowerThan95(y))
Output: If a student gets an A in English class, then his score is greater than 90 and lower than 95.
Target: If a student gets an A in English class, then his score is greater than 90 but lower than 95.
Distance: 3
BLEU: 0.9677315297684971

Input: There Exists x There Exists y(Act(x) and PreventsHarm(x) and (No (x=y)) and Act(y) and PreventsHarm(y))
Output: Some acts prevent harm.
Target: Some Acts prevent harmful things.
Distance: 11
BLEU: 0.6192690367626723

Input: OperatedUntil(yMCA, campDavern, year2015)
Output: Camp Davern was operated by the YMCA until the year 2015.
Target: Camp Davern was operated by the YMCA until the year 2015.
Distance: 0
BLEU: 0.9999824564481326

Input: Bioegradable(worksheet)
Output: A worksheet is biodegradable.
Target: A worksheet is biodegradable.
Distance: 0
BLEU: 0.9999655184303989

Figure 6: The first 5 examples of inputs (logical form), outputs (candidate) and targets (reference) in the Validation set.

FOL2NW: For All a (Mentors(a) and For All district IsSpacious(district) XOR There Exists president Mentors(president, specialist))
--> Districts are either spacious or have a president mentor a specialist.

FOL2NW: For All a (No Studies(a) XOR No Develops(a, bicycle) or No IsBasedIn(a, country))
--> A person either studies or does not develop a bicycle, or they are not based in a country.

FOL2NW: For All a (HasOfficeIn(a) implies For All chef (No LivesIn(chef, zone) or No IsThoughtful(chef)))
--> If the chef has an office in a building, then the chef neither lives in the zone nor is thoughtful.

FOL2NW: For All a Follows(a) implies For All architect No IsThoughtful(architect) and For All lawyer No Drives(lawyer)
--> If a person follows a lead, then no architect is thoughtful and no lawyer drives.

FOL2NW: For All a (TravelsTo(a) and There Exists man (No IsBusy(man) or There Exists teacher Uses (teacher, car)))
--> No man travels to another city if he is not busy or a teacher uses a car.

FOL2NW: For All a Endorses(a) implies There Exists region IsRural(region) and For All laboratory Endorses(laboratory)
--> If a person endorses a product, then the region is rural and the laboratory is also rural.

FOL2NW: For All a (No Roams(a, park) implies Roams(a))
--> If a person does not roam a park, then they roam.

FOL2NW: For All a No IsStudent(a) implies There Exists salesperson IsInnovative(salesperson) or There Exists researcher No IsAngry(researcher, d)
--> If A is not a student, then the salesperson is innovative or the researcher is not angry.

FOL2NW: For All a HasOfficeIn(a, state) and There Exists doctor No IsDoctor(doctor) or There Exists area No IsUrban(area, e)
--> A doctor has an office in the state, or the area is not urban.

FOL2NW: For All a (No IsInnovative(a) implies Endorses(a) XOR No IsSad(a, b))
--> Nobody who is not innovative endorses or is sad.

Figure 7: The first 10 examples of inputs (logical form) and outputs (candidate) in the Test set.